Scalability Audit of ShopSphere

CSA06 – Design and Analysis of Algorithms | CO1 – Assessment Tool 1 | Question 1

Student Name	_____
Student Reg. No.	_____
Course	CSA06 – Design and Analysis of Algorithms
Source code	`Source_Code/shopsphere_audit.py`

Conventions. log means $\log_2$. "Basic operation" = the operation whose count dominates the running time. The paper says the servers execute "10 basic operations per second", but the exponent was lost in the PDF; this report assumes 10^8 **operations/second** and shows in (g) that the conclusions are checked against 10^7 and 10^9 as well. The nightly window is 10 min = 600 s, i.e. a budget of 6×10^{10} operations at 10^8 ops/s.

(a) Understanding the Problem

Module	Input size	Basic operation	Where it occurs	Expected growth
M1 Duplicate detector	n = number of products	Title + seller comparison (`IF Product[i]... = Product[j]...`)	Innermost statement, executed once per pair (i, j), $i < j$	Quadratic: each product is compared with all later ones, so $\approx n^2/2$
M2 API rate limiter	n = number of requests (window limit)	`Audit(Request[j])`	Body of the `FOR j` loop; the loop runs i times where i doubles each pass	Linear: the geometric series $1 + 2 + 4 + ...$ is dominated by its last term, so $\approx n$ (not $n \log n$)
M3 Product search	n = catalogue size	Key comparison with a catalogue entry	S1: each scanned entry. S2: each probe of the middle entry	S1 linear; S2 logarithmic (the search range halves each step)
M4 Recommendation engine	n = size of the customer–product data	Basic work is the linear merge ($f(n) = n$) at each recursion node; the count is governed by the recurrence	Combine step after the recursive calls	Current $4T(n/2)$: quadratic. Redesign $2T(n/2)$: $n \log n$

Module	Input size	Basic operation	Where it occurs	Expected growth
M5 Sales analytics	n = number of sales records	Record comparison/move while sorting at each level, $f(n) = n \log n$	Sort at every recursion level	Slightly above $n \log n$; each of $\log n$ levels costs $n \log n$ at the top, giving $n \log^2 n$
M5 report generator	n = records	Elementary step counted in $f(n)$	Whole routine	$3n^2$ term dominates, so quadratic

Justification (brief). In M1 and M2 the dominant operation is the one in the innermost loop body, because every other statement (loop control, `i ← i × 2`) executes fewer times. In M3 the key comparison is the only operation repeated with the data size. In M4 and M5 the work outside the recursive calls (n and $n \log n$) is the repeated cost that the recurrence adds up.

(b) Analysing the Iterative Modules

M1 – Duplicate-Listing Detector

Basic operation: the comparison inside the inner loop. For a fixed i, j runs from $i+1$ to $n-1$, which is $n-1-i$ iterations. Summing over $i = 0 \ldots n-2$:

$$C_1(n) = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{(n-1)n}{2} = \frac{n^2 - n}{2}.$$

- Time complexity: $C_1(n) = \frac{1}{2}n^2 - \frac{1}{2}n = \Theta(n^2)$. The count is identical in best, average and worst case, because the loops do not depend on the data (`MarkDuplicate` is called at most once per pair and is not the dominant operation).
- Space complexity: only i, j and a counter are used besides the input, so $\Theta(1)$ extra space.
- Check: $n = 1000 \Rightarrow 499{,}500$ comparisons (program: 499,500).

M2 – API Rate Limiter

In the t-th pass ($t = 0, 1, 2, \ldots$) the value of i is 2^t, and the inner loop executes i times. The WHILE condition $i < n$ holds as long as $2^t < n$, i.e. $t \le \lceil \log_2 n \rceil - 1$. So the outer loop runs $\lceil \log_2 n \rceil$ times and

$$C_2(n) = \sum_{t=0}^{\lceil \log_2 n \rceil - 1} 2^t = 2^{\lceil \log_2 n \rceil} - 1.$$

Since $n \le 2^{\lceil \log_2 n \rceil} < 2n$ we get $n - 1 \le C_2(n) \le 2n - 1$. For n a power of two, $C_2(n) = n - 1$.

Is the Θ(n log n) claim correct? No. The team member multiplied (number of outer iterations) × (maximum inner iterations), which is only an upper bound, $O(n \log n)$. The inner loop runs up to n times only in the **last** pass; in the earlier passes it runs $n/2, n/4, \ldots$ times, which form a geometric series that sums to about n, not to $n \log n$. Formally, $n - 1 \le C_2(n) \le 2n - 1$ gives

$$C_2(n) = \Theta(n) \quad \text{(tight bound)},$$

and $C_2(n) \neq \Omega(n \log n)$ because $C_2(n)/(n \log n) \leq 2/\log n \to 0$. The claim is a valid (loose) O bound but not a Θ bound.

Verification, n = 1024. i takes the values $1, 2, 4, 8, 16, 32, 64, 128, 256, 512$ (10 passes; $i = 1024$ fails $i < n$):

$$1 + 2 + 4 + \cdots + 512 = 2^{10} - 1 = 1023.$$

The formula gives $2^{\lceil \log_2 1024 \rceil} - 1 = 2^{10} - 1 = 1023$, and the program counts 1023. The claimed $n \log n = 1024 \cdot 10 = 10{,}240$ is ten times too large; $1023/1024 \approx 0.999 \approx n$.

- Space complexity of M2: $\Theta(1)$.

(c) Recurrence for Search Version S2 – Backward Substitution

$$T(n) = T(n/2) + 1, \qquad T(1) = 1, \qquad n = 2^k.$$

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &= \big[T(n/4) + 1\big] + 1 = T(n/2^2) + 2 \\ &= \big[T(n/8) + 1\big] + 2 = T(n/2^3) + 3 \\ &\;\;\vdots \\ &= T(n/2^i) + i \quad \text{(general step, } i \text{ substitutions)}. \end{aligned}$$

The recursion stops at $n/2^i = 1$, i.e. $i = \log_2 n = k$. Then

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n.$$

Exact closed form: $T(n) = \log_2 n + 1$ for $n = 2^k$ (in general $\lfloor \log_2 n \rfloor + 1$). Check by induction: $T(1) = \log_2 1 + 1 = 1$; if $T(n/2) = \log_2 n$ then $T(n) = \log_2 n + 1$. Program check: $n = 2^{20} \Rightarrow T = 21$.

Time complexity: $T(n) = \Theta(\log n)$. For $n = 10^6$ at most 20 comparisons.

Implementation	Time	Extra space
Iterative binary search (`lo`, `hi`, `mid`)	$\Theta(\log n)$	$\Theta(1)$
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$, because the call stack reaches depth $\lfloor \log_2 n \rfloor + 1$ (measured: 20 for $n = 10^6$)

(d) Applying the Master Theorem

For $T(n) = aT(n/b) + f(n)$ compare $f(n)$ with $n^{\log_b a}$:

1. $f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow T = \Theta(n^{\log_b a})$.
2. $f(n) = \Theta(n^{\log_b a}) \Rightarrow T = \Theta(n^{\log_b a} \log n)$.
3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and regularity $\Rightarrow T = \Theta(f(n))$.

M4 – Recommendation engine

	Current design $T(n) = 4T(n/2) + n$	Redesign $T(n) = 2T(n/2) + n$
a	4	2
b	2	2
$f(n)$	n	n
$n^{\log_b a}$	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	$f(n) = n = O(n^{2-\varepsilon})$ with $\varepsilon = 1$	$f(n) = \Theta(n) = \Theta(n^{\log_b a})$
Case	**Case 1**	**Case 2**
Result	$\Theta(n^2)$	$\Theta(n \log n)$

Exact solutions for $n = 2^k$ (verified by program): current $T(n) = 2n^2 - n$; redesign $T(n) = n \log_2 n + n$.

Effect of doubling the data.

- Current: $T(2n)/T(n) \to 4$ (quadratic), so doubling the data quadruples the work (measured 4.00).
- Redesign: $T(2n)/T(n) = \dfrac{2n(\log n + 1) + 2n}{n \log n + n} \to 2$ from above, a little over double (measured 2.1–2.2 for $n = 2^{10}$–2^{18}).

So removing replication reduces the exponent from 2 to essentially 1.

M5 – Sales analytics: $T(n) = 2T(n/2) + n \log n$

$a = 2,\ b = 2,\ f(n) = n \log n,\ n^{\log_b a} = n.$

- Case 1 needs $f(n) = O(n^{1-\varepsilon})$: $n \log n$ grows faster than n, so no.
- Case 3 needs $f(n) = \Omega(n^{1+\varepsilon})$ for some $\varepsilon > 0$: $n \log n$ is smaller than $n^{1+\varepsilon}$ for any $\varepsilon > 0$, so no.
- Case 2 needs $f(n) = \Theta(n)$: $n \log n \neq \Theta(n)$, so no.

None of the three standard cases applies. The function lies in the gap between n and $n^{1+\varepsilon}$.

Extended Master Theorem ($f(n) = \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$ gives $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$): here $k = 1$, so $T(n) = \Theta(n \log^2 n)$.

Recursion-tree check. Level i has 2^i nodes of size $n/2^i$, each costing $\frac{n}{2^i} \log \frac{n}{2^i}$, so the level costs $n(\log n - i)$:

$$T(n) = \sum_{i=0}^{k-1} n(k-i) + n \cdot T(1) = n\frac{k(k+1)}{2} + n, \qquad k = \log_2 n.$$

This equals $\frac{1}{2} n \log^2 n + \frac{1}{2} n \log n + n = \Theta(n \log^2 n)$ (program-verified for $n = 2^1 \ldots 2^{16}$).

(e) Best, Average and Worst Cases

	Best case	Average case	Worst case
S1 (sequential)	Key in first position: **1** comparison	See derivation below	Key last, or absent: n comparisons
S2 (binary)	Key equals the middle entry on the first probe: **1** comparison	Successful: about $\log_2 n - 1$; unsuccessful: about $\log_2 n$, i.e. $\Theta(\log n)$	Key found at the last probe or absent: $\lfloor \log_2 n \rfloor + 1$ comparisons

For S2 the exact successful-search average over $n = 2^k - 1$ equally likely keys is $\dfrac{(k-1)2^k + 1}{2^k - 1}$ (program: 9.0098 for $n = 1023$). For $n = 10^6$: best 1, worst 20.

S1 average with probability p of the key being present

If the product is present, it is at each of the n positions with probability $1/n$ and costs i comparisons at position i. Absent, it costs n comparisons.

$$A(n,p) = p \cdot \frac{1}{n}\sum_{i=1}^{n} i + (1-p)\,n = p \cdot \frac{n+1}{2} + (1-p)\,n.$$

For $n = 1{,}000{,}000$ and $p = 0.5$:

$$A = 0.5 \cdot \frac{1{,}000{,}001}{2} + 0.5 \cdot 1{,}000{,}000 = 250{,}000.25 + 500{,}000 = \mathbf{750{,}000.25} \text{ comparisons.}$$

Special cases: $p = 1 \Rightarrow (n+1)/2 = 500{,}000.5$; $p = 0 \Rightarrow n = 1{,}000{,}000$. Overall $A = \Theta(n)$, against at most 20 for S2.

(f) Proving an Asymptotic Bound

$f(n) = 3n^2 + 10n \log n + 50$. We must find $c_1, c_2, n_0 > 0$ with $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

Lower bound. Since $10n \log n + 50 > 0$ for $n \ge 2$, $f(n) \ge 3n^2$. So $c_1 = 3$, valid for all $n \ge 1$.

Upper bound. For $n \ge 1$: $\log n \le n$, so $10n \log n \le 10n^2$; and $50 \le 50n^2$. Hence

$$f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2, \qquad c_2 = 63,\ n_0 = 1.$$

A tighter pair: we want $10n \log n + 50 \le n^2$, giving $f(n) \le 4n^2$. At $n = 63$: $10 \cdot 63 \cdot 5.977 + 50 = 3815.9 \le 3969$ (holds). At $n = 64$: $10 \cdot 64 \cdot 6 + 50 = 3890 \le 4096$ (holds), and the gap $n^2 - 10n \log n$ keeps growing after $n = 64$ (checked by program up to 10^6). At $n = 32$ the inequality fails ($1650 > 1024$), so $n_0 = 64$ is a convenient choice.

Result: $3n^2 \le f(n) \le 4n^2$ for all $n \ge n_0 = 64$ (alternatively $c_2 = 63$, $n_0 = 1$). Therefore

$$f(n) = \Theta(n^2). \qquad \blacksquare$$

f(n) is not O(n). Suppose, for contradiction, that constants $c > 0$ and n_0 exist with $f(n) \le cn$ for all $n \ge n_0$. Since $f(n) \ge 3n^2$, this would require $3n^2 \le cn$, i.e. $n \le c/3$ for all $n \ge n_0$. This fails

for every $n > \max(n_0, c/3)$, a contradiction. Hence $f(n) \neq O(n)$. (Numerically $f(n)/n$ = 68 at $n = 10$, 3,100 at $n = 10^3$, 300,166 at $n = 10^5$: unbounded.)

(g) Scalability Decision

Assumed speed 10^8 ops/s. Window = 600 s = 6×10^{10} operations. Counts use the exact formulas derived above, with $\log_2 10^6 = 19.93$ (M5 uses $\lfloor \log_2 n \rfloor = 19$, so it is an estimate for a non-power of two).

Module	Basic operations at $n = 10^6$	Time at 10^8 ops/s	Fits 10 min?
M1 duplicate detector, $n(n-1)/2$	499,999,500,000 ($\approx 5 \times 10^{11}$)	83.3 min	**NO**
M2 rate limiter, $2^{\lceil \log n \rceil} - 1$	1,048,575	10.5 ms	Yes
S2 binary search (one query)	20	negligible	Yes
M4 current, $2n^2 - n$	1,999,999,000,000 ($\approx 2 \times 10^{12}$)	5.56 h	**NO**
M4 redesign, $n \log n + n$	20,931,568 ($\approx 2.1 \times 10^7$)	0.21 s	Yes
M5 analytics, $\frac{1}{2} n \log^2 n$	$\approx 1.91 \times 10^8$	1.9 s	Yes

At today's $n = 10^5$: M1 takes 50 s and M4 (current) 3.3 min, both inside the window. Over the year's growth ($n \times 10$) both quadratic modules grow by **100×**, M2 by about 8×, M4-redesign by about 12×.

Sensitivity to server speed ($n = 10^6$).

Module	10^7 ops/s	10^8 ops/s	10^9 ops/s
M1	13.9 h – fail	83.3 min – fail	8.3 min – OK (barely)
M4 current	2.3 days – fail	5.6 h – fail	33 min – fail
M4 redesign, M5, M2, S2	OK	OK	OK

Modules that fail the 10-minute window: M1 and M4 (current design) at the assumed speed. M4's current design fails at every speed tested; M1 passes only if the servers reach 10^9 ops/s, with almost no headroom.

Recommendations

1. **M4 – must be redesigned.** Adopt the no-replication design, $T(n) = 2T(n/2) + n = \Theta(n \log n)$. That cuts 2×10^{12} operations to 2.1×10^7 (about 10^5 times fewer) and brings the run from hours to a fraction of a second.
2. **M1 – must be redesigned.** Pairwise comparison is unnecessary. Insert each product's key (title, seller) into a hash set and mark a duplicate when the key is already present: expected $\Theta(n)$ time, $\Theta(n)$ extra space, about 10^6 operations instead of 5×10^{11}. If memory is the concern, sort by (title, seller) in $\Theta(n \log n)$ and compare neighbours.
3. **M2 – keep.** It is $\Theta(n)$ (about n audits), not $n \log n$; there is no scalability risk.

4. **M3 – use S2.** Use the sorted index (about 20 comparisons) rather than S1 (about 750,000 on average for $p = 0.5$); keep the index sorted on insertion.
5. **M5 – acceptable, optimisable.** $\Theta(n \log^2 n)$ is fine at this scale (about 2 s). Because sorting at every level is redundant, a single merge-sort-style pass would reduce it to $\Theta(n \log n)$ if the data grows much further.
6. **Report generator, $f(n) = \Theta(n^2)$.** Watch it: if it processes $n = 10^6$ items it needs about 3×10^{12} operations (about 8 h at 10^8 ops/s).

Appendix – Source Code and Output Screenshots

The program `Source_Code/shopsphere_audit.py` implements instrumented M1, M2, S1, S2 (iterative and recursive), the M4/M5 recurrences, and the scalability table. Each closed form in this report is asserted against the counted value.

Source code

Program output

shopsphere_audit.py (lines 1-55)

```python
"""
CSA06 - Design and Analysis of Algorithms
CO1 - Assessment Tool 1: Scalability Audit of "ShopSphere"

Implements instrumented versions of modules M1-M5 and S1/S2, counts the basic
operation, checks every closed form derived in the report against the counts,
and produces the scalability table for n = 100,000 and n = 1,000,000.

Run:  python3 shopsphere_audit.py
"""
import math
import random

# ---------------------------------------------------------------------------
# Assumption: the question paper prints "10 basic operations per second" with
# the exponent lost in the PDF. We assume 10^8 ops/s (typical for one server
# core) and also print a sensitivity table for 10^7 and 10^9.
# ---------------------------------------------------------------------------
OPS_PER_SEC = 10 ** 8
WINDOW_SEC = 10 * 60          # nightly window: 10 minutes


# ============================== (b) M1 ========================================
def m1_count(n):
    """Run the M1 double loop, counting the title/seller comparisons."""
    count = 0
    for i in range(0, n - 1):           # i = 0 .. n-2
        for j in range(i + 1, n):       # j = i+1 .. n-1
            count += 1                  # one comparison (basic operation)
    return count


def m1_formula(n):
    return n * (n - 1) // 2


# ============================== (b) M2 ========================================
def m2_count(n):
    """Run the M2 loop, counting Audit() calls."""
    count = 0
    i = 1
    while i < n:
        for _ in range(0, i):
            count += 1                  # Audit(Request[j])
        i *= 2
    return count


def m2_formula(n):
    """Exact count = 2^ceil(log2 n) - 1 (n >= 2)."""
    return 2 ** math.ceil(math.log2(n)) - 1


# ============================== (c)/(e) M3 ====================================
def s1_search(arr, key):
```

Figure 1: Source code, part 1

shopsphere_audit.py (lines 56-110)

```
    """Sequential search; returns (index or -1, comparisons)."""
    comps = 0
    for idx, v in enumerate(arr):
        comps += 1
        if v == key:
            return idx, comps
    return -1, comps


def s2_iterative(arr, key):
    """Binary search (iterative); returns (index or -1, comparisons)."""
    lo, hi, comps = 0, len(arr) - 1, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        comps += 1
        if arr[mid] == key:
            return mid, comps
        elif arr[mid] < key:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, comps


def s2_recursive(arr, key, lo=0, hi=None, depth=1):
    """Binary search (recursive); returns (index, comparisons, max_depth)."""
    if hi is None:
        hi = len(arr) - 1
    if lo > hi:
        return -1, 0, depth
    mid = (lo + hi) // 2
    if arr[mid] == key:
        return mid, 1, depth
    if arr[mid] < key:
        idx, c, d = s2_recursive(arr, key, mid + 1, hi, depth + 1)
    else:
        idx, c, d = s2_recursive(arr, key, lo, mid - 1, depth + 1)
    return idx, c + 1, d


def t_s2(n):
    """T(n) = T(n/2) + 1, T(1) = 1, evaluated directly (n a power of 2)."""
    return 1 if n == 1 else t_s2(n // 2) + 1


def s1_average(n, p):
    """E[comparisons] = p(n+1)/2 + (1-p)n."""
    return p * (n + 1) / 2 + (1 - p) * n


# ============================ (d) M4 / M5 ================================
def t_m4_current(n):
    return 1 if n == 1 else 4 * t_m4_current(n // 2) + n


```

Figure 2: Source code, part 2

shopsphere_audit.py (lines 111-165)

```
def t_m4_redesign(n):
    return 1 if n == 1 else 2 * t_m4_redesign(n // 2) + n


def t_m5(n):
    return 1 if n == 1 else 2 * t_m5(n // 2) + n * int(math.log2(n))


def m4_current_closed(n):          # 2n^2 - n
    return 2 * n * n - n


def m4_redesign_closed(n):         # n log n + n
    return n * int(math.log2(n)) + n


def m5_closed(n):                  # n*k(k+1)/2 + n, k = log2 n
    k = int(math.log2(n))
    return n * k * (k + 1) // 2 + n


# ============================ (f) ============================================
def f(n):
    return 3 * n * n + 10 * n * math.log2(n) + 50


def check_theta_n2():
    """Verify 3n^2 <= f(n) <= 4n^2 for all n >= 64 (checked up to 10^6),
    and also f(n) <= 63 n^2 for all n >= 1."""
    ok_tight = all(3 * n * n <= f(n) <= 4 * n * n for n in range(64, 10 ** 6 + 1, 7))
    ok_loose = all(f(n) <= 63 * n * n for n in range(2, 10 ** 5 + 1, 3))
    return ok_tight, ok_loose


# ============================ main ===========================================
def fmt_time(sec):
    if sec < 1:
        return f"{sec * 1000:.3f} ms"
    if sec < 120:
        return f"{sec:.2f} s"
    if sec < 7200:
        return f"{sec / 60:.2f} min"
    if sec < 172800:
        return f"{sec / 3600:.2f} h"
    return f"{sec / 86400:.2f} days"


def main():
    print("=" * 72)
    print(" ShopSphere Scalability Audit  -  CSA06 CO1-AT1")
    print("=" * 72)
    print(f"Assumed server speed : {OPS_PER_SEC:.0e} basic operations / second")
    print(f"Nightly window       : {WINDOW_SEC} s (10 minutes)\n")

    # ---- (b) M1 verification
```

Figure 3: Source code, part 3

shopsphere_audit.py (lines 166-220)

```
    print("-- (b) M1: comparisons = n(n-1)/2 -----------------------------------")
    for n in (2, 5, 10, 100, 1000):
        c, fm = m1_count(n), m1_formula(n)
        print(f"n={n:>5}  counted={c:>8}  formula={fm:>8}  match={c == fm}")
        assert c == fm

    # ---- (b) M2 verification
    print("\n-- (b) M2: Audit calls = 2^ceil(log2 n) - 1 --------------------------")
    for n in (2, 3, 5, 8, 100, 1000, 1024, 1025):
        c, fm = m2_count(n), m2_formula(n)
        print(f"n={n:>5}  counted={c:>6}  formula={fm:>6}  "
              f"claim n*log2(n)={n * math.log2(n):>8.0f}  match={c == fm}")
        assert c == fm
    n = 1024
    print(f"\nVerification n=1024: i takes 1,2,4,...,512 (10 values); "
          f"sum = {m2_count(n)} = 2^10 - 1 = {2**10 - 1}")
    print(f"Team-member claim n*log2(n) = {n * int(math.log2(n))}  "
          f"(10x too large)   ratio count/n = {m2_count(n) / n:.4f}")

    # ---- (c) S2 recurrence
    print("\n-- (c) S2: T(n) = T(n/2)+1, T(1)=1  ->  T(n) = log2 n + 1 -----------")
    for k in range(0, 21, 4):
        n = 2 ** k
        print(f"n=2^{k:<2} = {n:>8}  T(n)={t_s2(n):>3}  log2 n + 1 = {k + 1:>3}")
        assert t_s2(n) == k + 1

    # ---- (c)/(e) search measurements
    print("\n-- (e) Search cases (n = 1,000,000 sorted keys 0..n-1) --------------")
    N = 1_000_000
    arr = list(range(N))
    _, s1_best = s1_search(arr, 0)
    _, s1_worst = s1_search(arr, N - 1)
    _, s1_miss = s1_search(arr, -5)
    print(f"S1 best  (key at position 1)    : {s1_best} comparison")
    print(f"S1 worst (last / absent)        : {s1_worst} / {s1_miss} comparisons")
    # key at 0-based position idx costs idx+1 comparisons; average over all N keys
    avg_present = sum(idx + 1 for idx in range(N)) / N
    print(f"S1 avg, successful (all keys)   : {avg_present:.1f}  (theory (n+1)/2 = {(N + 1) / 2:.1f})")
    print(f"S1 avg, p=0.5 (formula)         : {s1_average(N, 0.5):.2f}")
    _, b_best = s2_iterative(arr, (N - 1) // 2)
    worst_b = max(s2_iterative(arr, k)[1] for k in random.sample(range(N), 20000))
    _, miss_b = s2_iterative(arr, -5)
    _, _, depth = s2_recursive(arr, -5)
    print(f"S2 best (key = middle)          : {b_best} comparison")
    print(f"S2 worst (sampled / absent)     : {worst_b} / {miss_b}  "
          f"(floor(log2 n)+1 = {int(math.log2(N)) + 1})")
    print(f"S2 recursion depth (absent key) : {depth}  -> recursive stack O(log n)")
    # exact S2 average for successful search, n = 2^k - 1
    k = 10
    m = 2 ** k - 1
    a = list(range(m))
    tot = sum(s2_iterative(a, x)[1] for x in a)
    print(f"S2 avg successful, n={m}: {tot / m:.4f}  "
          f"(formula ((k-1)2^k+1)/(2^k-1) = {((k - 1) * 2 ** k + 1) / m:.4f})")
    print("S1 avg for p in {0, 0.5, 1} at n = 10^6: "
```

Figure 4: Source code, part 4

shopsphere_audit.py (lines 221-275)

```
          + ", ".join(f"{s1_average(N, p):,.2f}" for p in (0, 0.5, 1)))

    # ---- (d) recurrences
    print("\n-- (d) M4 / M5 recurrences vs closed forms --------------------------")
    for k in (1, 4, 8, 12, 16):
        n = 2 ** k
        a, b, c = t_m4_current(n), t_m4_redesign(n), t_m5(n)
        assert a == m4_current_closed(n) and b == m4_redesign_closed(n) and c == m5_closed(n)
        print(f"n=2^{k:<2}  M4cur={a:>12,}  (2n^2-n)  M4new={b:>10,}  (n log n + n)  "
              f"M5={c:>12,}  (n k(k+1)/2 + n)")
    print("\nDoubling the data (n -> 2n):")
    for k in (10, 14, 18):
        n = 2 ** k
        print(f"n=2^{k}: M4 current x{m4_current_closed(2 * n) / m4_current_closed(n):.3f}   "
              f"M4 redesign x{m4_redesign_closed(2 * n) / m4_redesign_closed(n):.3f}   "
              f"M5 x{m5_closed(2 * n) / m5_closed(n):.3f}")

    # ---- (f) asymptotic proof check
    print("\n-- (f) f(n) = 3n^2 + 10 n log2 n + 50  is  Theta(n^2) ----------------")
    t, l = check_theta_n2()
    print(f"3n^2 <= f(n) <= 4n^2 for all tested n >= 64  : {t}")
    print(f"f(n) <= 63 n^2 for all tested n >= 2         : {l}")
    for n in (8, 16, 32, 63, 64, 128):
        print(f"n={n:>4}  f(n)={f(n):>10.0f}  4n^2={4 * n * n:>8}  f<=4n^2 : {f(n) <= 4 * n * n}")
    print("f(n)/n  grows without bound (not O(n)):",
          ", ".join(f"n={n}: {f(n) / n:,.0f}" for n in (10, 1000, 100000)))

    # ---- (g) scalability table
    print("\n-- (g) Scalability decision ------------------------------------------")
    rows = []
    for N in (100_000, 1_000_000):
        lg = math.log2(N)
        ops = {
            "M1  n(n-1)/2": m1_formula(N),
            "M2  2^ceil(lg n)-1": m2_formula(N),
            "S2  floor(lg n)+1 (one query)": int(lg) + 1,
            "M4 current 2n^2-n": 2 * N * N - N,
            "M4 redesign n lg n + n": int(N * lg + N),
            "M5  n lg^2 n / 2": int(N * int(lg) * (int(lg) + 1) / 2 + N),
        }
        print(f"\nn = {N:,}   (log2 n = {lg:.3f})")
        print(f"{'Module':<32}{'basic ops':>20}{'time @1e8/s':>16}{'<=10 min?':>11}")
        for name, o in ops.items():
            sec = o / OPS_PER_SEC
            print(f"{name:<32}{o:>20,}{fmt_time(sec):>16}{'YES' if sec <= WINDOW_SEC else 'NO':>11}")
            rows.append((N, name, o, sec))

    print("\nSensitivity of the 10-minute verdict to the assumed server speed (n = 10^6):")
    N = 1_000_000
    lg = int(math.log2(N))
    base = {"M1": m1_formula(N), "M2": m2_formula(N), "M4 current": 2 * N * N - N,
            "M4 redesign": int(N * math.log2(N) + N),
            "M5": N * lg * (lg + 1) // 2 + N}
    print(f"{'Module':<14}" + "".join(f"{'@' + format(r, '.0e') + '/s':>18}" for r in (1e7, 1e8, 1e9)))
    for name, o in base.items():
```

Figure 5: Source code, part 5

shopsphere_audit.py (lines 276-289)

```
        print(f"{name:<14}" + "".join(
            f"{fmt_time(o / r) + (' OK' if o / r <= WINDOW_SEC else ' FAIL'):>18}" for r in (1e7, 1e8, 1e9)))

    print("\nGrowth when n goes 10^5 -> 10^6 (x10):")
    print(f"  M1 x{m1_formula(10**6) / m1_formula(10**5):.1f}   "
          f"M2 x{m2_formula(10**6) / m2_formula(10**5):.1f}   "
          f"M4 current x{(2e12 - 1e6) / (2e10 - 1e5):.1f}   "
          f"M4 redesign x{base['M4 redesign'] / (1e5 * math.log2(1e5) + 1e5):.1f}")
    print("\nAll assertions passed.")


if __name__ == "__main__":
    main()

```

Figure 6: Source code, part 6

```
$ python3 shopsphere_audit.py  (part 1)

======================================================================
 ShopSphere Scalability Audit  -  CSA06 CO1-AT1
======================================================================
Assumed server speed : 1e+08 basic operations / second
Nightly window       : 600 s (10 minutes)

-- (b) M1: comparisons = n(n-1)/2 -----------------------------------
n=    2  counted=       1  formula=       1  match=True
n=    5  counted=      10  formula=      10  match=True
n=   10  counted=      45  formula=      45  match=True
n=  100  counted=    4950  formula=    4950  match=True
n= 1000  counted=  499500  formula=  499500  match=True

-- (b) M2: Audit calls = 2^ceil(log2 n) - 1 -------------------------
n=    2  counted=     1  formula=     1  claim n*log2(n)=       2  match=True
n=    3  counted=     3  formula=     3  claim n*log2(n)=       5  match=True
n=    5  counted=     7  formula=     7  claim n*log2(n)=      12  match=True
n=    8  counted=     7  formula=     7  claim n*log2(n)=      24  match=True
n=  100  counted=   127  formula=   127  claim n*log2(n)=     664  match=True
n= 1000  counted=  1023  formula=  1023  claim n*log2(n)=    9966  match=True
n= 1024  counted=  1023  formula=  1023  claim n*log2(n)=   10240  match=True
n= 1025  counted=  2047  formula=  2047  claim n*log2(n)=   10251  match=True

Verification n=1024: i takes 1,2,4,...,512 (10 values); sum = 1023 = 2^10 - 1 = 1023
Team-member claim n*log2(n) = 10240  (10x too large)   ratio count/n = 0.9990

-- (c) S2: T(n) = T(n/2)+1, T(1)=1  ->  T(n) = log2 n + 1 -----------
n=2^0  =        1  T(n)=  1  log2 n + 1 =   1
n=2^4  =       16  T(n)=  5  log2 n + 1 =   5
n=2^8  =      256  T(n)=  9  log2 n + 1 =   9
n=2^12 =     4096  T(n)= 13  log2 n + 1 =  13
n=2^16 =    65536  T(n)= 17  log2 n + 1 =  17
n=2^20 =  1048576  T(n)= 21  log2 n + 1 =  21

-- (e) Search cases (n = 1,000,000 sorted keys 0..n-1) --------------
S1 best  (key at position 1)    : 1 comparison
S1 worst (last / absent)        : 1000000 / 1000000 comparisons
S1 avg, successful (all keys)   : 500000.5  (theory (n+1)/2 = 500000.5)
S1 avg, p=0.5 (formula)         : 750000.25
S2 best (key = middle)          : 1 comparison
S2 worst (sampled / absent)     : 20 / 19  (floor(log2 n)+1 = 20)
S2 recursion depth (absent key) : 20  -> recursive stack O(log n)
S2 avg successful, n=1023: 9.0098  (formula ((k-1)2^k+1)/(2^k-1) = 9.0098)
S1 avg for p in {0, 0.5, 1} at n = 10^6: 1,000,000.00, 750,000.25, 500,000.50

-- (d) M4 / M5 recurrences vs closed forms --------------------------
n=2^1   M4cur=           6  (2n^2-n)  M4new=         4  (n log n + n)  M5=           4  (n k(k+1)/2 + n)
n=2^4   M4cur=         496  (2n^2-n)  M4new=        80  (n log n + n)  M5=         176  (n k(k+1)/2 + n)
n=2^8   M4cur=     130,816  (2n^2-n)  M4new=     2,304  (n log n + n)  M5=       9,472  (n k(k+1)/2 + n)
n=2^12  M4cur=  33,550,336  (2n^2-n)  M4new=    53,248  (n log n + n)  M5=     323,584  (n k(k+1)/2 + n)
```

Figure 7: Output, part 1

```
$ python3 shopsphere_audit.py  (part 2)

n=2^16  M4cur=8,589,869,056  (2n^2-n)  M4new= 1,114,112  (n log n + n)  M5=   8,978,432  (n k(k+1)/2 + n)

Doubling the data (n -> 2n):
n=2^10: M4 current x4.001   M4 redesign x2.182   M5 x2.393
n=2^14: M4 current x4.000   M4 redesign x2.133   M5 x2.283
n=2^18: M4 current x4.000   M4 redesign x2.105   M5 x2.221

-- (f) f(n) = 3n^2 + 10 n log2 n + 50  is  Theta(n^2) ----------------
3n^2 <= f(n) <= 4n^2 for all tested n >= 64  : True
f(n) <= 63 n^2 for all tested n >= 2         : True
n=   8  f(n)=       482  4n^2=     256  f<=4n^2 : False
n=  16  f(n)=      1458  4n^2=    1024  f<=4n^2 : False
n=  32  f(n)=      4722  4n^2=    4096  f<=4n^2 : False
n=  63  f(n)=     15723  4n^2=   15876  f<=4n^2 : True
n=  64  f(n)=     16178  4n^2=   16384  f<=4n^2 : True
n= 128  f(n)=     58162  4n^2=   65536  f<=4n^2 : True
f(n)/n  grows without bound (not O(n)): n=10: 68, n=1000: 3,100, n=100000: 300,166

-- (g) Scalability decision ------------------------------------------

n = 100,000   (log2 n = 16.610)
Module                                  basic ops     time @1e8/s  <=10 min?
M1  n(n-1)/2                        4,999,950,000         50.00 s        YES
M2  2^ceil(lg n)-1                        131,071        1.311 ms        YES
S2  floor(lg n)+1 (one query)                  17        0.000 ms        YES
M4 current 2n^2-n                   19,999,900,000        3.33 min        YES
M4 redesign n lg n + n                   1,760,964       17.610 ms        YES
M5  n lg^2 n / 2                        13,700,000      137.000 ms        YES

n = 1,000,000   (log2 n = 19.932)
Module                                  basic ops     time @1e8/s  <=10 min?
M1  n(n-1)/2                      499,999,500,000       83.33 min         NO
M2  2^ceil(lg n)-1                        1,048,575       10.486 ms        YES
S2  floor(lg n)+1 (one query)                  20        0.000 ms        YES
M4 current 2n^2-n                 1,999,999,000,000          5.56 h         NO
M4 redesign n lg n + n                   20,931,568      209.316 ms        YES
M5  n lg^2 n / 2                        191,000,000          1.91 s        YES

Sensitivity of the 10-minute verdict to the assumed server speed (n = 10^6):
Module                 @1e+07/s          @1e+08/s          @1e+09/s
M1                 13.89 h FAIL     83.33 min FAIL       8.33 min OK
M2                104.858 ms OK      10.486 ms OK       1.049 ms OK
M4 current        2.31 days FAIL       5.56 h FAIL    33.33 min FAIL
M4 redesign            2.09 s OK     209.316 ms OK      20.932 ms OK
M5                    19.10 s OK         1.91 s OK     191.000 ms OK

Growth when n goes 10^5 -> 10^6 (x10):
  M1 x100.0   M2 x8.0   M4 current x100.0   M4 redesign x11.9

All assertions passed.
```

Figure 8: Output, part 2